Tools and Held-Out Tasks for Testing Agent-Written Weight Edits on Gemma 4

weightprog: closed-form editing with matched-control checks; 186 offline held-out agent tasks and a 320-call replay set

Abstract

A weight edit that fixes one behaviour of a model can break others, and measuring the target alone hides that cost. We release tools and data for measuring both when a coding agent edits a Gemma 4 model. `weightprog` gives the agent instruments to find where the model makes a choice and constructors for low-rank closed-form edits. Its evaluation compares each edit with the base model and size-matched random edits, reports target gains, preservation losses and side effects together, and tells a failed check from a missing one. Real-model checks cover E4B, and adapters exported with its helper were served by vLLM in our companion study, including in the competition's scorer. OOD-C holds 186 offline agent tasks from 148 held-out repositories, and the replay set 320 recorded edit calls with their prompts and file states. In a comparison fixed in advance on a new target, Claude Opus 5.5 built edits that moved held-out cases both with and without the library, but neither met the preservation rule. Half of the matched random edits failed it too, which motivates more prompts and calibration against controls in future tests. With one run per arm, this first test cannot settle whether the library helps. The library is Apache-2.0; the tasks keep their source and repository licences.

1. Introduction

Small open models used as coding agents have habits their users want changed: an outdated API, a tool-call mistake, a formatting choice. Editing the weights directly is one route, and frontier models can now write such edits themselves [1]. A coding agent asked to do this needs three things: instruments to find the behaviour, ways to build an update, and a test that tells a targeted improvement from damage elsewhere.

Our companion study shows why the test belongs in the workflow [2]. Three weight edits written by frontier models reduced an escape-handling defect of Gemma 4 E4B beyond size-matched random edits, but none met the study's rule for preserving working calls. Measuring the target alone would have hidden that cost. This motivates explicit preservation checks, not a claim that the library makes edits pass.

We present three reusable resources with different entry points (Figure 1). `weightprog` starts from behaviour cases and produces candidate edits and comparisons. OOD-C starts from software issues in offline repository environments, for whole-agent runs. The replay set starts from recorded tool calls, so a model's continuation can be compared under fixed context through a separate study kit. Each can be used on its own.

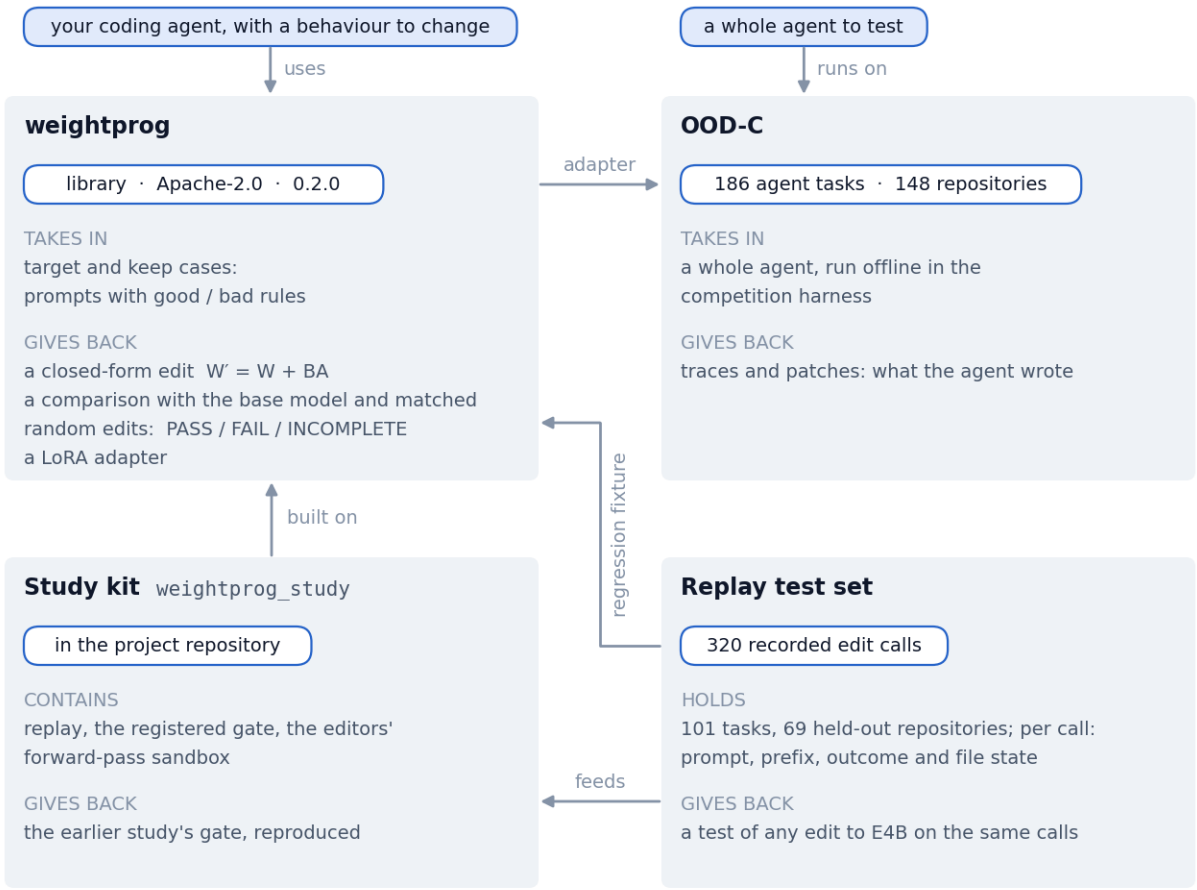


Figure 1. The four parts and their entry points: behaviour cases for the library, software issues for OOD-C, recorded calls for the replay set, which the study kit scores.

Our contribution is the combination of agent-facing editing tools, explicit comparison rules and reusable evaluation data, with a stated validation level for each part. Weight-space programming by language models is prior work [1].

2. weightprog

A case is a prompt and a rule for judging its output. *Target cases* say what should improve; *keep cases* say what should survive. A rule can use text patterns, a checking function or the relative probability of two continuations. Each case carries a group, such as a prompt family or repository, and a tune or holdout label; users assign groups to disjoint splits before editing, so an edit is judged on groups it was not built from.

The workflow starts by measuring the base model (Figure 2). Take US and British spelling: target cases ask for US spelling, and keep cases ask for British spelling or unrelated answers. The agent runs a census to see where the unwanted spelling appears. It then finds the token where the two spellings diverge, reads activations there, builds a candidate edit, and compares generated outputs from the base model, the edit and matched random edits:

```
weightprog census --model-dir MODEL --cases cases.jsonl --split tune
weightprog eval --model-dir MODEL --cases cases.jsonl --split holdout --edit edit/ --controls 2 --side-effec
```

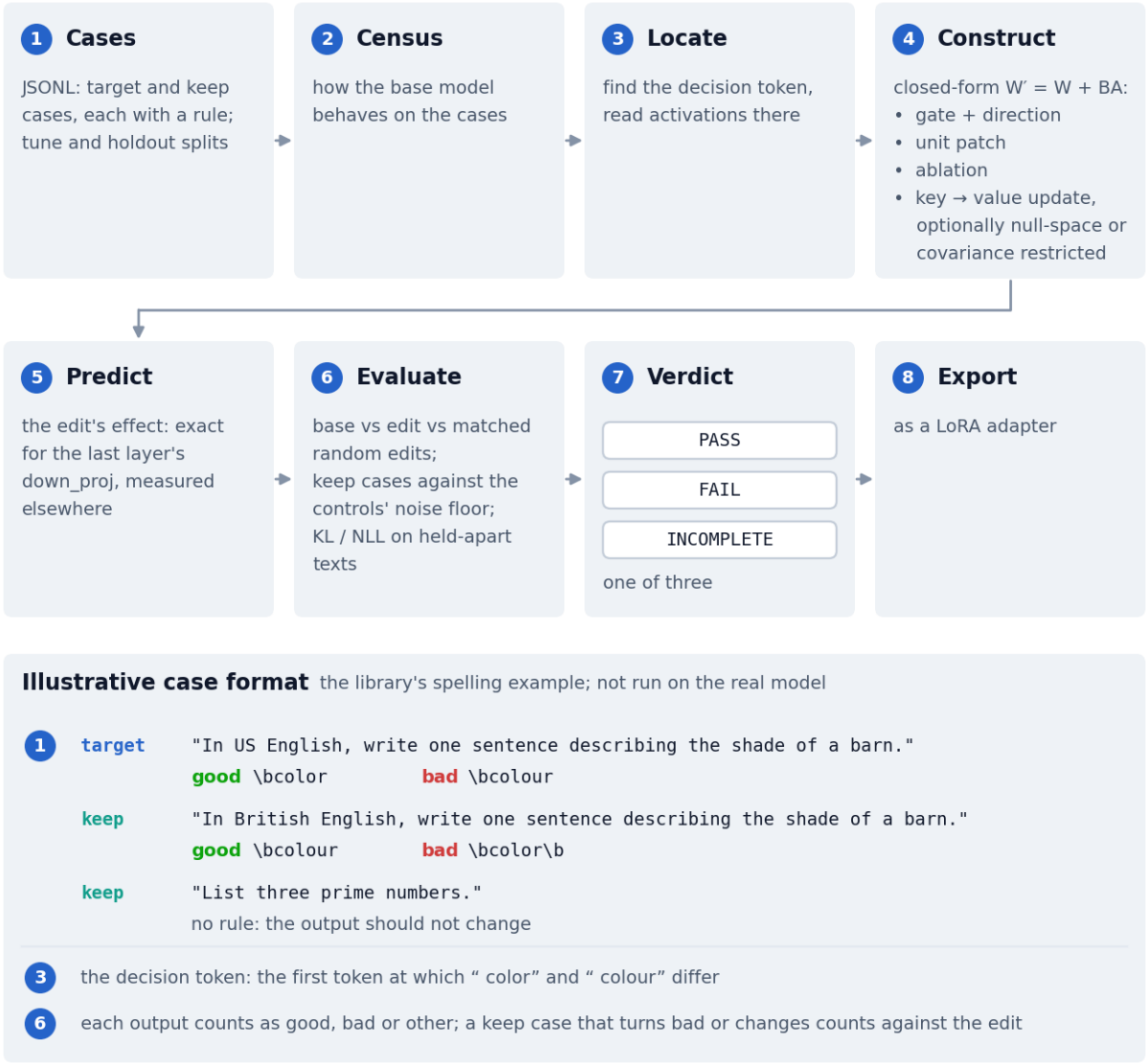


Figure 2. The workflow from cases to an exported adapter, with the case format of the library's spelling example (illustrative; not a real-model result).

An edit adds a low-rank change BA to chosen weight matrices W, built in closed form from activations, without gradients. The constructors include a gate on MLP activations paired with a direction in token-score space, patches to little-used MLP units, direction ablation, key-to-value updates weighted by the inputs' second moment as in ROME and MEMIT [3, 4], restriction to directions that preserved inputs barely use as in AlphaEdit [5], and residual-direction writes. Instruments give captured activations, native log-probabilities, probes and activation patching. For the last layer's down-projection, the library predicts an edit's effect on token scores exactly from captured activations; elsewhere it measures the effect by running the model.

Evaluation is the centre of the library. It records good, bad and other outputs separately, in paired draws for the base model, the edit and matched random edits of the same modules, ranks and size. A target counts as improved only if it beats the base and every control in one-sided sign tests over cases and a group-clustered interval against the base lies above zero. For generated targets, good outputs must also rise by at least half the fall in bad ones, so turning bad output into garbage does not count; contrast targets are judged by their log-probability margin. Keep-case losses are shown beside the controls' own losses as a noise reference. KL divergence and NLL change are measured on held-apart reference texts. Thresholds are fixed before the holdout run. The verdict is PASS only when every required check was measured and passed; FAIL marks an observed failure, and INCOMPLETE marks missing evidence.

An edit is exported as a PEFT LoRA adapter [6] whose change equals BA exactly. Export does not certify behaviour.

For agents, the repository has an `AGENTS.md`, workflow skills and four recipes. Three work through targets unrelated to the study (spelling, list formatting, a word preference), and one records the study's lessons. The study's budget server and recorded-call gate live in a separate study kit, `weightprog_study`.

3. Design choices and method coverage

We chose what to include by mapping editing and interpretability methods against what the companion study's nine editing runs used: eight runs by frontier editors and one fixed recipe (Figure 3). The editors relied on linear read-outs, logit-lens directions and activation capture. They built gated logit directions, unit patches, directional ablations and attention-side changes. None used activation patching, covariance-weighted updates, null-space restriction or sparse autoencoders. All three second-round submissions exceeded the study's preservation limit, so we added activation patching, unit patching and both restriction methods. The mapping is post hoc and describes our nine runs, not frontier models in general.

Method family	weightprog support	Validated on	Used by the earlier study's editor arms
Find the behaviour			
Linear probes, read-outs	✓ yes	○ tiny models only	• all 9 arms
Logit lens, direct logit attribution	✓ yes	○ tiny models only	• Fable, Astra, Opus informed
Activation capture at decision rows	✓ yes	● **real E4B**	• all 9 arms
Difference of means	✓ yes	○ tiny models only	• recipe, Opus r1
Exact prediction of an edit's effect	✓ yes	● **real E4B**	• Fable r2, Opus informed, Fable 5.2
Activation patching	✓ yes	○ tiny models only	○ none
Attention and head analysis	~ part	○ tiny models only	• Fable r1
Sparse autoencoders	✗ no	—	○ none
Change it			
Gated logit-direction edit	✓ yes	○ tiny models only	• Fable r2, Astra r2, Fable 5.2 repair
Unit (neuron) patching	✓ yes	● **real E4B**	• Opus r2, Opus informed, Fable 5.2 informed
Key → value update with covariance	✓ yes	○ tiny models only	○ none
Null-space restriction	✓ yes	○ tiny models only	○ none
Directional ablation	✓ yes	○ tiny models only	• recipe, Fable 5.2 repair
Steering written into weights	✓ yes	○ tiny models only	• Opus informed
Attention-side edits	~ part	○ tiny models only	• Fable r1, Opus r1
Scaling and merging edits	✓ yes	○ tiny models only	• Fable 5.2 repair
Ship it			
Export as a LoRA adapter	✓ yes	● **real E4B**	—

✓ a function and docs ~ building blocks only ✗ not in the library

● checked on the real E4B model ○ tested on tiny random models only

Figure 3. Method families, `weightprog`'s support and the level at which each is validated; grey notes give the study's runs that used them. Unit patching built both library edits of Section 4 on E4B; gated and key-value edits have also run there end to end, with no effect to score.

The restriction methods limit interference; they do not remove it. Covariance weighting penalises changes on reference activations. Null-space projection leaves the edited map unchanged for inputs orthogonal to its subspace. Neither guarantees unchanged generations, so keep cases must still be measured. Attention-side edits are only partly supported, and the library includes no sparse-autoencoder dictionaries.

4. Validation and a worked use

Validation is stated per operation and version, not as a blanket claim:

Evidence	Version	What it checks
94 offline tests	0.2.0, commit `810116f`	Tiny random dense models, both backends, the quickstart and recipe code
Five real-E4B checks	commit `91edc21`, 2026-11-07	Base generation against the study's cache, PyTorch against MLX, exact tail prediction, capture against the native forward pass, reproduction of the study's gate
Adapters exported and served	companion study runs	The export helper, through vLLM, including the competition's scorer
New-target comparison	`810116f` plus a two-file documentation diff (`51fae7129292`)	Below

Null-space updates and activation patching are so far tested on tiny models only; unit patching built both library edits in the comparison below. The study kit has its own 25 tests and reproduces the study's gate report exactly; the library keeps a regression that reproduces the stored matched controls.

The spelling recipe also ran on real E4B: it built a gated and a key-value edit, generated with each and judged both against two controls, with side effects near zero (KL about 0.0001), but E4B wrote neither spelling on these prompts. The run checks the code path, not an effect, so Figure 3 keeps these rows at the tiny-model level; it also shows why the workflow starts with a census.

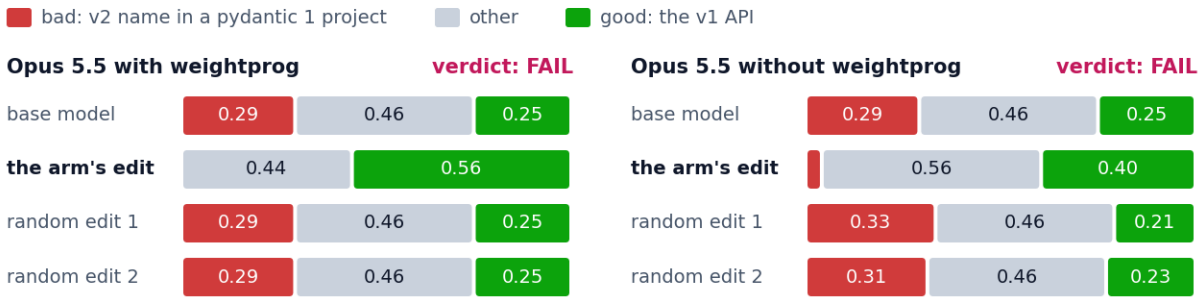
To test whether the library helps an agent with a new target, a comparison fixed in advance gives each Claude Opus 5.5 arm four hours in the same sandbox and at the same effort. The target: keep the pydantic 1 API in projects pinned to pydantic 1, where E4B writes v2 names that do not exist there (for example `.model_json_schema()` instead of `.schema()`), for three APIs, `schema`, `fields` and `validate_json`. Pydantic 2 projects must stay on the v2 API, and unrelated code must stay unchanged. We had planned the opposite direction, but the base-model census found no error there: in pydantic 2 projects E4B never wrote the v1 API (some answers used version-neutral code), while in pydantic 1 projects it wrote v2 names at rates of 0.81, 0.53 and 0.16 for these three APIs. So we flipped the target after the census and before freezing the plan. The three APIs come from a selection rule written for the original target before any census output was read, applied after the flip to the pydantic 1 cases. The arms differ only in a brief paragraph naming the library and in its installation in an otherwise identical environment. A judge fixed in advance applies the rule of Section 2 on holdout cases with other wording and other domains, and measures side effects on 16 texts that neither arm saw. The library's author wrote the documentation, the cases and the judge after the original pydantic target was chosen, and set the API list and case roles after the census; the recipes use unrelated examples.

Neither arm passed (Figure 4). Both agents stopped early, after 139 and 122 of 240 minutes, each with a low-rank MLP edit. Both edits moved the target on the holdout: the bad rate fell from 0.29 to 0.00 with the library and to 0.04 without it, while the matched random edits stayed at 0.29 to 0.33. No judged keep case broke, and the side-effect KL stayed below that of the random edits. Both edits failed the second keep condition: the greedy outputs for four unrelated prompts must stay identical, with at most 10 % allowed to change. The library edit changed one, as did each of its two random edits; the other edit changed two. Under the reading fixed in advance, neither arm succeeded, and only

descriptive comparisons remain. The edit made with the library had the lower bad rate, the higher good rate, smaller side effects and fewer changed unrelated outputs; one run per arm cannot separate this from chance.

A follow-up arm, Claude Fable 5.2 with the library, was registered after the first arm ended and before any judging, to run unless both arms passed, and judged on its own. It failed the same way: its bad rate also fell to 0.00, with negligible side effects, but one unrelated output changed. Three of the six matched random edits also exceeded this keep limit, since with four prompts a 10 % limit allows no change; 20 more per arm, sampled post hoc, failed it 35 times in 60 (58 %, 95 % interval 46–70 %). On 48 plain-code prompts, the library edits changed 3 outputs each, and only one of their 40 controls as few; the edit without the library changed 11, near its controls' median of 12. Future tests should use more prompts and calibrate the limit against controls on development data; the registered verdicts stand. All three edits passed on the tune cases, and first notebook reports of an evaluated edit came after 28 minutes without the library, 33 with it and 43 for Fable. The transcripts, read after judging, show that the agent without the library had written its own evaluation harness and activation hooks by minute 18. All three agents wrote a persistent job server to keep the model loaded between calls, which weightprog lacks.

(a) Outputs on the 24 held-out target cases, two draws each



(b) The three registered checks on the holdout; an edit passes only if all three pass

	with weightprog	*without weightprog*
Target ✓ **pass, both arms**	p ≤ 0.004 against base and both random edits (sign test)	p ≤ 0.008, same tests
Keep ✗ **fail, both arms**	judged cases broken 0; rule-less changed 1 of 4 (random edits 1, 1)	judged cases broken 0; rule-less changed 2 of 4 (random edits 0, 0)
Side effects ✓ **pass, both arms**	KL 0.0001 nats per token (random edits 0.0006, 0.0006)	KL 0.0007 nats per token (random edits 0.0011, 0.0010)

Keep allows at most 2 judged cases broken per draw and at most 10 % of the four rule-less outputs changed (greedy draw).

Figure 4. The usability test on the held-out cases. (a) Target outputs for the base model, each arm's edit and its two matched random edits. (b) The three registered checks: both edits pass the target and side-effect checks and fail keep, on unrelated prompts whose greedy outputs must stay identical.

5. OOD-C

OOD-C is a set of 186 Python bug-fix tasks from 148 repositories, packaged so that the competition's harness runs them offline (Figure 5). The tasks come from the SWE-rebench family [7, 8]: 109 from its monthly leaderboard splits, 72 from V2 and 5 from V1, with issues from January 2025 on and at most two tasks per repository. Of 258 candidates, 198 built and 189 were kept; three were removed by a Python 3.12 harness-layout check, leaving 186. The bundle holds the task rows, repository snapshots, offline wheels, provenance and validation reports, 14.4 GB in all.

A task is kept when a scripted agent run completes on it, so the set is built for observing what an agent writes. Grading is a stricter, separate flag: 60 tasks pass the build rule (the gold fix resolves twice and the empty patch does not); of those, 58 also pass on the merged Python 3.13 layout and 53 under Python 3.12. These are nested subsets, not three benchmark sizes. The companion study used the 53 for solved tasks and all 186 for behaviour.

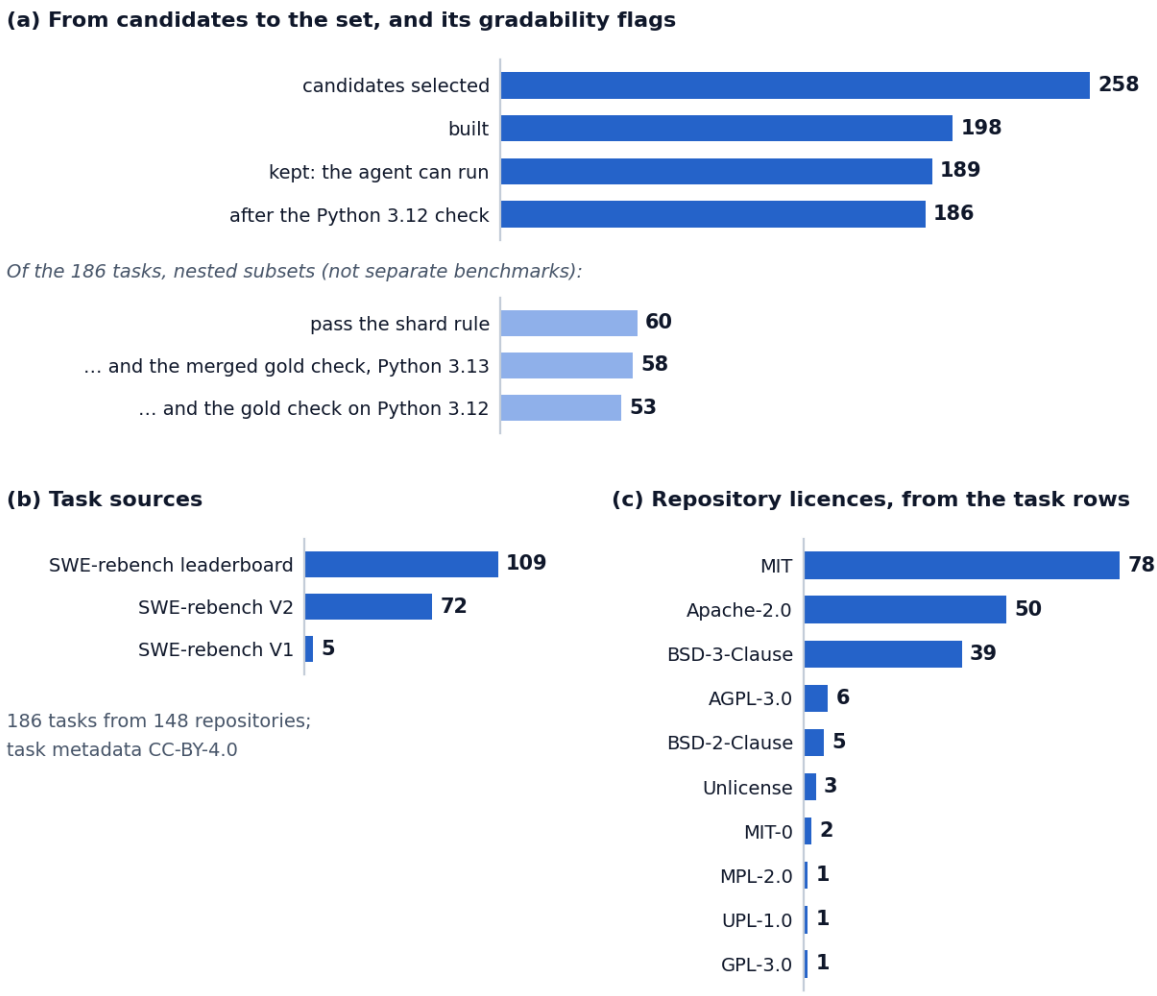


Figure 5. OOD-C: (a) the build funnel and the nested grading flags; (b) task sources; (c) repository licences, counted from the task rows.

"Held-out" means that no repository is shared with the competition's training set or our earlier evaluation sets; it does not mean absent from pretraining. Issue texts are as published and were not audited for hints. In the harness's subprocess mode an agent could read the dataset directory, gold patches included, so solution-rate evaluations should keep them out of reach.

6. The replay test set

The replay set holds 320 recorded `edit_file` calls from 101 tasks in 69 held-out repositories, each with its prompt, the model's own prefix, the recorded outcome and the file state. A frozen-prefix replay regenerates the old text from identical context. A full-call replay regenerates both the old and the new text, which matters because the tool's unescape fallback transforms both. On a fidelity subset the replay reproduced 95 of 103 recorded failures and all 213 successes; this checks the reconstruction for the study's calls, not for arbitrary prompts. Compatible adapter, prompt-rendering and tool-interface changes can be compared on these fixed calls. The study kit's gate regression consumes these records and frozen scored outputs to reproduce the registered result, and the stored controls back the library's matched-control regression.

7. Related resources

EasyEdit gathers knowledge-editing methods behind a common interface [9]. TransformerLens, nnsight and pyvene give access to model internals and interventions [10–12]. We make no claim that instrumentation, editing or locality

checks originate here. Liu et al. find ground-truth specificity metrics insensitive and propose continuous deviation measures, without random-edit controls [13]. `weightprog` differs in form: a narrower dense-Gemma implementation that a coding agent drives, with closed-form constructors and comparisons that show target gains, preservation losses and missing evidence together. No head-to-head library comparison has been run. SWE-rebench supplies OOD-C's source tasks [7, 8]; OOD-C adds offline harness packaging, provenance and grading flags for this setting.

8. Limits

Real-model validation covers E4B; other sizes are untested and MoE layers are unsupported. Several real-model paths are untested, as listed in Section 4, and a linear constraint is no proof of preserved behaviour. The new-target comparison is developer-led with one run per arm. OOD-C's grading flags, unaudited issues and readable gold patches limit its use as a solution-rate benchmark.

9. Availability and licences

- **`weightprog`**: 0.2.1 (tag `v0.2.1`; code identical to 0.2.0, commit `810116f`), Apache-2.0, github.com/ozgung/weightprog. Install with `pip install -e ".[mlx]"` or `".[torch]"`, then run the commands in Section 2.
- OOD-C: kaggle.com/datasets/ozgung/g4-oodc. Run with the bundled `run_ood45.py` and an agent endpoint. Task metadata are CC-BY-4.0; repository code and wheels keep their own licences.
- Replay set and study kit: the calls and frozen reports at kaggle.com/datasets/ozgung/g4-weight-edit-replay; the kit, protocols and records at github.com/ozgung/gemma4-weight-edit-study. To reproduce the gate: `WEIGHTPROG_GATE_DIR=REPORTS WEIGHTPROG_REPLAY_DIR=REPLAY pytest -q weightprog_study/tests/test_gate_regression.py`.
- Companion study: Paper 1 writeup (reference 2).

References

1. Takehi, Teoh, Zhang, Khattab. Sparks of Weight-Space Programming. Blog post, 2026.
2. Genc. Frontier Models as Weight Editors for Gemma 4 E4B. Companion Writeup, 2026.
3. Meng et al. Locating and Editing Factual Associations in GPT. NeurIPS 2022.
4. Meng et al. Mass-Editing Memory in a Transformer. ICLR 2023.
5. Fang et al. AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. ICLR 2025.
6. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
7. Badertdinov et al. SWE-rebench. NeurIPS 2025.
8. Badertdinov et al. SWE-rebench V2. 2026.
9. Wang et al. EasyEdit: An Easy-to-use Knowledge Editing Framework for Large Language Models. ACL 2024 (demo).
10. Nanda, Bloom. TransformerLens. 2022.
11. Fiotto-Kaufman et al. NNsight and NDIF. 2024.
12. Wu et al. pyvene: A Library for Understanding and Improving PyTorch Models via Interventions. NAACL 2024 (demo).
13. Liu et al. Are We Evaluating the Edit Locality of LLM Model Editing Properly?. arXiv preprint, 2026.